WebSocket (Socket.io) Reference Guide

WebSocket (Socket.io) Reference Guide

> **Audience:** JavaScript developers building real-time apps with Socket.io.

---

1) What is a WebSocket, and how it differs from HTTP

Mental model: *HTTP is like sending letters. WebSockets are like staying on a phone call.*

- **HTTP (request/response)**
  - The client sends a request.
  - The server sends a response.
  - Connection is usually short-lived.
  - Great for pages and APIs that do not need instant updates.
- **WebSocket (persistent connection)**
  - The client and server establish a connection once.
  - After that, either side can send messages any time.
  - Great for chat, multiplayer games, live dashboards, collaborative editing.

How Socket.io relates: Socket.io *uses WebSockets when possible*, and can fall back to other transports in tricky environments. It also adds a higher-level API: events, rooms, automatic reconnection, acknowledgements, etc.

---

2) Setup (Express + Socket.io) — server boilerplate

Mental model: *You attach Socket.io to an HTTP server, then listen for connections and events.*

Backend (Node.js)

```
// server.js
import express from "express";
import http from "http";
import { Server } from "socket.io";

const app = express();

// Why: Socket.io needs the raw HTTP server so it can upgra
de HTTP -> WebSocket.
const server = http.createServer(app);

// Why: io is the "server-side socket hub". It manages all
sockets, rooms, and broadcasts.
const io = new Server(server, {
  cors: {
    // Why: browsers block cross-origin websocket requests
unless allowed.
    origin: "http://localhost:3000",
    methods: ["GET", "POST"],
  },
});

// Why: Express is still useful for normal HTTP routes (hea
lth checks, REST endpoints, static files).
app.get("/health", (req, res) => {
  res.json({ ok: true });
});

const PORT = 4000;
server.listen(PORT, () => {
  console.log(`HTTP + WebSocket server listening on :${POR
T}`);
});

export { io };
```

Frontend (React)

```
// The client connects to the same server URL where Socket.
io is running.
// You will build this out in the client section.
```

---

3) Core events and core APIs (with paired frontend + backend examples)

Mental model: *On the server you react to connections and events. On the client you connect and react to events.*

> In all examples below, assume you already created `io` on the backend, and `socket` on the frontend.

A) `io.on("connection")` (reserved)

Mental model: *This is the "new caller joined the phone call" event.*

- **What it is:** Fires every time a client successfully connects.
- **Why it is reserved:** Socket.io needs this exact event name to deliver the new socket instance.

Backend

```
io.on("connection", (socket) => {
  console.log("New client connected:", socket.id);

  // You typically register per-socket listeners here.
  socket.on("ping", () => {
    socket.emit("pong");
  });
});
```

Frontend

```
socket.emit("ping");

socket.on("pong", () => {
```

```
  console.log("Server replied: pong");
});
```

B) `socket.on("disconnect")` (reserved)

Mental model: *The call dropped for this specific person.*

- **What it is:** Fires when this socket disconnects.
- **Why it is reserved:** Socket.io owns the connection lifecycle and emits this automatically.

Backend

```
io.on("connection", (socket) => {
  socket.on("disconnect", (reason) => {
    console.log("Client disconnected:", socket.id, "reaso
n:", reason);
  });
});
```

Frontend

```
socket.on("disconnect", (reason) => {
  console.log("Disconnected:", reason);
});
```

C) `socket.emit()` (send to one user)

Mental model: *Talk to just this one connected client.*

Backend

```
io.on("connection", (socket) => {
  socket.emit("welcome", { message: "Welcome!", yourSocketI
d: socket.id });
});
```

Frontend

```
socket.on("welcome", (payload) => {
  console.log(payload.message, payload.yourSocketId);
});
```

D) `socket.on()` (listen for a custom event)

Mental model: *You create your own event names for your app protocol.*

Frontend

```
socket.emit("chat:send", { roomId: "room-1", text: "hello"
});
```

Backend

```
io.on("connection", (socket) => {
  socket.on("chat:send", ({ roomId, text }) => {
    console.log("Got chat message:", roomId, text);
  });
});
```

E) `io.emit()` (send to all users)

Mental model: *Announcement over the loudspeaker to everyone connected.*

Backend

```
function broadcastSystemMessage(text) {
  io.emit("system:message", { text });
}

broadcastSystemMessage("Server will restart in 5 minutes");
```

Frontend

```
socket.on("system:message", ({ text }) => {
  console.log("SYSTEM:", text);
```

```
});
```

F) `io.to(room).emit()` (send to everyone in a room)

Mental model: *Send a message to everyone inside a specific group chat.*

Backend

```
io.to("room-1").emit("room:notice", { text: "Hello room-1"
});
```

Frontend

```
socket.on("room:notice", ({ text }) => {
  console.log("ROOM NOTICE:", text);
});
```

G) `socket.to(room).emit()` (send to everyone in room except sender)

Mental model: *Tell everyone else in the room, but not yourself.*

Backend

```
io.on("connection", (socket) => {
  socket.on("typing:start", ({ roomId }) => {
    socket.to(roomId).emit("typing:status", { socketId: soc
ket.id, isTyping: true });
  });
});
```

Frontend

```
socket.emit("typing:start", { roomId: "room-1" });

socket.on("typing:status", ({ socketId, isTyping }) => {
  console.log(socketId, "typing?", isTyping);
});
```

H) `socket.join(room)` (join a room)

Mental model: *Put this socket into a named group so you can broadcast to the group later.*

Backend

```
io.on("connection", (socket) => {
  socket.on("room:join", async ({ roomId }) => {
    await socket.join(roomId);

    // Useful confirmation back to the joiner
    socket.emit("room:joined", { roomId });

    // Notify others already in the room
    socket.to(roomId).emit("room:memberJoined", { socketId:
socket.id });
  });
});
```

Frontend

```
socket.emit("room:join", { roomId: "room-1" });

socket.on("room:joined", ({ roomId }) => {
  console.log("Joined room:", roomId);
});

socket.on("room:memberJoined", ({ socketId }) => {
  console.log("Someone joined:", socketId);
});
```

I) `socket.off()` (stop listening) + React cleanup

Mental model: *If you do not unhook listeners, you can accidentally subscribe multiple times.*

- In React, components can mount and unmount.

- If you call `socket.on(...)` on every mount without cleanup, you can get duplicate handlers.

Frontend (React)

```
import { useEffect } from "react";

export function useRoomNotice(socket) {
  useEffect(() => {
    function onRoomNotice(payload) {
      console.log("room notice:", payload);
    }

    socket.on("room:notice", onRoomNotice);

    return () => {
      // Remove ONE specific handler
      socket.off("room:notice", onRoomNotice);

      // Alternative: remove all handlers for an event
      // socket.off("room:notice");
    };
  }, [socket]);
}
```

Backend (context)

```
// Server just emits "room:notice" as shown earlier.
io.to("room-1").emit("room:notice", { text: "example" });
```

---

4) Reserved vs custom event names

Mental model: *Reserved events are owned by Socket.io. Everything else is your app protocol.*

Common reserved events (do not redefine)

- `connection` (server side, on `io`)
- `connect` (client side)

- `disconnect` (both sides, socket lifecycle)
- `disconnecting`
- `connect_error`
- `reconnect`, `reconnect_attempt`, `reconnect_error` (depending on client version)

Good custom event naming conventions

- Use **namespaces** with colons, like `room:join`, `game:answer`, `chat:send`.
- Keep payloads JSON-serializable.
- Prefer "verb" names for actions: `game:start`, `quiz:submitAnswer`.

Backend

```
socket.on("game:start", ({ roomId }) => {
  // custom
});
```

Frontend

```
socket.emit("game:start", { roomId: "abc" });
```

---

5) Rooms concept

Mental model: *A room is a label that lets you broadcast to a subset of sockets.*

Why rooms exist

- You rarely want to broadcast everything to everyone.
- Rooms let you scope messages to:
  - one game lobby
  - one chat channel
  - one collaborative document

2-player scenario (room holds exactly 2 players)

Backend (tracking occupancy)

```
io.on("connection", (socket) => {
  socket.on("room:join", async ({ roomId }) => {
    await socket.join(roomId);

    const room = io.sockets.adapter.rooms.get(roomId);
    const size = room ? room.size : 0;

    io.to(roomId).emit("room:count", { roomId, count: size
});

    if (size === 2) {
      io.to(roomId).emit("game:ready", { roomId });
    }

    if (size > 2) {
      // Optional: enforce 2 players
      socket.emit("room:error", { message: "Room is full"
});
      socket.leave(roomId);
    }
  });
});
```

Frontend

```
socket.emit("room:join", { roomId: "match-123" });

socket.on("room:count", ({ count }) => {
  console.log("Players in room:", count);
});

socket.on("game:ready", () => {
  console.log("We have 2 players. Start UI.");
});

socket.on("room:error", ({ message }) => {
```

```
  alert(message);
});
```

6) `emit` vs `on` — the rule (with a two-column diagram)

Mental model: `emit` *sends an event.* `on` *receives an event.*

Frontend (React)	Backend (Node + Socket.io)
1) `socket.emit("room:join", payload)`	2) `socket.on("room:join", handler)`
4) `socket.on("room:joined", handler)`	3) `socket.emit("room:joined", payload)`

Example pair

Frontend

```
socket.emit("room:join", { roomId: "room-1" });

socket.on("room:joined", ({ roomId }) => {
  console.log("confirmed:", roomId);
});
```

Backend

```
socket.on("room:join", async ({ roomId }) => {
  await socket.join(roomId);
  socket.emit("room:joined", { roomId });
});
```

7) Client-side setup — `socket.io-client` in React (with cleanup)

Mental model: *Create one socket connection, then attach listeners inside* `useEffect` *and clean them up.*

Frontend (React)

```
// socket.js
import { io } from "socket.io-client";
```

```
export const socket = io("http://localhost:4000", {
  transports: ["websocket"],
  autoConnect: true,
});
```

```
// App.jsx
import { useEffect, useState } from "react";
import { socket } from "./socket";

export default function App() {
  const [connected, setConnected] = useState(socket.connect
ed);

  useEffect(() => {
    function onConnect() {
      setConnected(true);
    }

    function onDisconnect() {
      setConnected(false);
    }

    socket.on("connect", onConnect);
    socket.on("disconnect", onDisconnect);

    return () => {
      socket.off("connect", onConnect);
      socket.off("disconnect", onDisconnect);
    };
  }, []);

  return <div>{connected ? "connected" : "disconnected"}</d
iv>;
}
```

Backend (minimal)

```
io.on("connection", (socket) => {
  console.log("client connected", socket.id);
});
```

---

8) Full quiz app flow (2 players)

Mental model: *Room is the match. Server is the referee. Clients are the players.*

Events (app protocol)

- `match:join` (client -> server)
- `match:state` (server -> room)
- `quiz:start` (server -> room)
- `quiz:answer` (client -> server)
- `quiz:result` (server -> room)
- `match:playerLeft` (server -> room)

Backend (Node + Socket.io)

```
// quizServer.js
import express from "express";
import http from "http";
import { Server } from "socket.io";

const app = express();
const server = http.createServer(app);
const io = new Server(server, {
  cors: { origin: "http://localhost:3000", methods: ["GET",
"POST"] },
});

// In-memory match state (for learning). Use Redis / DB for
production.
const matches = new Map();
// matches.get(roomId) => { players: Set<socketId>, answer
```

```
s: Map<socketId, string>, started: boolean }

function getOrCreateMatch(roomId) {
  if (!matches.has(roomId)) {
    matches.set(roomId, {
      players: new Set(),
      answers: new Map(),
      started: false,
    });
  }
  return matches.get(roomId);
}

io.on("connection", (socket) => {
  socket.on("match:join", async ({ roomId }) => {
    const match = getOrCreateMatch(roomId);

    // Enforce 2 players
    if (match.players.size >= 2) {
      socket.emit("match:error", { message: "Match is full"
});
      return;
    }

    await socket.join(roomId);
    match.players.add(socket.id);

    // Send state update to the room
    io.to(roomId).emit("match:state", {
      roomId,
      playerCount: match.players.size,
      started: match.started,
    });

    // Start when exactly 2 players
    if (match.players.size === 2 && !match.started) {
      match.started = true;
      match.answers.clear();
```

```
      io.to(roomId).emit("quiz:start", {
        roomId,
        question: "What does HTTP stand for?",
        choices: [
          "HyperText Transfer Protocol",
          "High Transfer Text Process",
          "Hyperlink Transmission Program",
          "Home Tool Transfer Protocol",
        ],
      });
    }
  });

  socket.on("quiz:answer", ({ roomId, answer }) => {
    const match = matches.get(roomId);
    if (!match || !match.started) return;

    // Record answer
    match.answers.set(socket.id, answer);

    // When both answers arrive, broadcast results
    if (match.answers.size === 2) {
      const correct = "HyperText Transfer Protocol";

      const results = Array.from(match.answers.entries()).m
ap(([socketId, a]) => ({
        socketId,
        answer: a,
        correct: a === correct,
      }));

      io.to(roomId).emit("quiz:result", { roomId, results
});

      // Reset for next question (simple demo)
      match.answers.clear();
    } else {
      // Let the room know someone answered (without reveal
```

```
ing it)
      socket.to(roomId).emit("quiz:opponentAnswered", { soc
ketId: socket.id });
    }
  });

  socket.on("disconnect", () => {
    // Find which match they were in (simple scan for demo)
    for (const [roomId, match] of matches.entries()) {
      if (match.players.has(socket.id)) {
        match.players.delete(socket.id);
        match.answers.delete(socket.id);

        socket.to(roomId).emit("match:playerLeft", { socket
Id: socket.id });
        io.to(roomId).emit("match:state", {
          roomId,
          playerCount: match.players.size,
          started: match.started,
        });

        // Optional: cleanup empty matches
        if (match.players.size === 0) {
          matches.delete(roomId);
        } else {
          // Optional: if someone leaves, stop the match
          match.started = false;
          match.answers.clear();
        }
      }
    }
  });
});

server.listen(4000, () => console.log("Quiz server on :400
0"));
```

Frontend (React)

```
// QuizClient.jsx
import { useEffect, useMemo, useState } from "react";
import { io } from "socket.io-client";

export default function QuizClient() {
  const socket = useMemo(() => io("http://localhost:4000",
{ transports: ["websocket"] }), []);

  const [roomId, setRoomId] = useState("match-123");
  const [playerCount, setPlayerCount] = useState(0);
  const [question, setQuestion] = useState(null);
  const [choices, setChoices] = useState([]);
  const [results, setResults] = useState(null);

  useEffect(() => {
    function onMatchState({ playerCount }) {
      setPlayerCount(playerCount);
    }

    function onQuizStart({ question, choices }) {
      setQuestion(question);
      setChoices(choices);
      setResults(null);
    }

    function onQuizResult({ results }) {
      setResults(results);
    }

    function onPlayerLeft() {
      setQuestion(null);
      setChoices([]);
      setResults(null);
    }

    socket.on("match:state", onMatchState);
    socket.on("quiz:start", onQuizStart);
    socket.on("quiz:result", onQuizResult);
```

```
    socket.on("match:playerLeft", onPlayerLeft);

    return () => {
      socket.off("match:state", onMatchState);
      socket.off("quiz:start", onQuizStart);
      socket.off("quiz:result", onQuizResult);
      socket.off("match:playerLeft", onPlayerLeft);
      socket.disconnect();
    };
  }, [socket]);

  function join() {
    socket.emit("match:join", { roomId });
  }

  function sendAnswer(answer) {
    socket.emit("quiz:answer", { roomId, answer });
  }

  return (
    <div>
      <h2>Room: {roomId}</h2>
      <button onClick={join}>Join match</button>
      <div>Players: {playerCount}/2</div>

      {question && (
        <div>
          <h3>{question}</h3>
          {choices.map((c) => (
            <button key={c} onClick={() => sendAnswer(c)}>
              {c}
            </button>
          ))}
        </div>
      )}

      {results && (
        <pre>{JSON.stringify(results, null, 2)}</pre>
```

```
        )}
      </div>
    );
  }
```

---

9) Quick reference cheat sheet (emit variants)

Mental model: *Pick the narrowest audience that matches your intent.*

Targeting (server side)

- **One socket (the connected client)**
  - `socket.emit(event, payload)`
- **All sockets (everyone)**
  - `io.emit(event, payload)`
- **Everyone in a room**
  - `io.to(roomId).emit(event, payload)`
  - `io.in(roomId).emit(event, payload)` (alias)
- **Everyone in a room except the sender**
  - `socket.to(roomId).emit(event, payload)`
  - `socket.broadcast.to(roomId).emit(event, payload)`
- **Everyone except the sender (global)**
  - `socket.broadcast.emit(event, payload)`

Targeting (client side)

- **Client can only emit from itself**
  - `socket.emit(event, payload)`
- **With acknowledgement (callback confirmation)**
  - `socket.emit(event, payload, (ack) => { /* server replied */ })`

Listening (both sides)

- `socket.on(event, handler)`
- `socket.off(event, handler)`
- `socket.once(event, handler)` — fires only once, then auto-removes itself

---

10) Acknowledgements (callbacks)

Mental model: *Like a certified letter — you know the other side received it.*

- By default, `emit` is fire-and-forget. The sender has no idea if the receiver got the message.
- Acknowledgements let the receiver send a callback reply back to the sender.
- Great for: confirming an action happened, returning a result, or handling errors.

Backend

```
io.on("connection", (socket) => {
  socket.on("room:join", async ({ roomId }, callback) => {
    try {
      await socket.join(roomId);
      // Call the callback to acknowledge success
      callback({ success: true, roomId });
    } catch (err) {
      callback({ success: false, error: err.message });
    }
  });
});
```

Frontend

```
socket.emit("room:join", { roomId: "room-1" }, (ack) => {
  if (ack.success) {
    console.log("Joined room:", ack.roomId);
  } else {
    console.error("Failed to join:", ack.error);
  }
});
```

> **Why use this over a separate response event?** Acknowledgements are request-scoped. You don't need to match a response event to a specific request manually.

---

11) Middleware (authentication)

Mental model: *A bouncer at the door — runs before any socket connects.*

- Use `io.use(fn)` to intercept every new connection before it is accepted.
- The most common use case is verifying a JWT or session token.
- If you call `next(new Error(...))`, the connection is rejected and the client gets a `connect_error` event.

Backend

```
import jwt from "jsonwebtoken";

io.use((socket, next) => {
  // Token usually passed via handshake auth
  const token = socket.handshake.auth?.token;

  if (!token) {
    return next(new Error("Authentication required"));
  }

  try {
    const payload = jwt.verify(token, process.env.JWT_SECRE
T);
    // Attach user info to socket for use in event handlers
    socket.data.user = payload;
    next();
  } catch (err) {
    next(new Error("Invalid token"));
  }
});

io.on("connection", (socket) => {
  console.log("Authenticated user:", socket.data.user);
});
```

Frontend

```
import { io } from "socket.io-client";

const socket = io("http://localhost:4000", {
  auth: {
    token: localStorage.getItem("jwt"),
  },
});

// Handle rejected connection
socket.on("connect_error", (err) => {
  console.error("Connection refused:", err.message);
});
```

---

12) Namespaces

Mental model: *Separate channels on the same server — like different radio frequencies.*

- By default, everything runs on the `/` namespace.
- Namespaces let you split concerns: `/chat`, `/admin`, `/game` — each with independent event handling and middleware.
- A single Socket.io server can serve multiple namespaces without running multiple servers.

Backend

```
// Default namespace (what you've been using)
io.on("connection", (socket) => { /* ... */ });

// Custom namespace
const chatNsp = io.of("/chat");
chatNsp.on("connection", (socket) => {
  console.log("User joined /chat namespace:", socket.id);
  socket.on("message", ({ text }) => {
    chatNsp.emit("message", { from: socket.id, text });
  });
});
```

```
const adminNsp = io.of("/admin");
// You can attach separate middleware per namespace
adminNsp.use((socket, next) => {
  if (socket.handshake.auth?.adminKey === process.env.ADMIN
_KEY) {
    next();
  } else {
    next(new Error("Unauthorized"));
  }
});
adminNsp.on("connection", (socket) => {
  console.log("Admin connected:", socket.id);
});
```

Frontend

```
import { io } from "socket.io-client";

// Connect to a specific namespace
const chatSocket = io("http://localhost:4000/chat");
const adminSocket = io("http://localhost:4000/admin", {
  auth: { adminKey: "secret" },
});

chatSocket.on("message", ({ from, text }) => {
  console.log(`${from}: ${text}`);
});
```

> **Namespaces vs Rooms:** Namespaces are server-defined and require the client to connect to them explicitly. Rooms are server-side only groupings within a namespace — clients never "connect" to a room directly.